



Chapitre 6 – La modularité en Java (Approche procédurale)

Par Robert DIASSÉ



Introduction

Lorsque les programmes deviennent plus longs et plus complexes, écrire tout le code dans la méthode `main` pose plusieurs problèmes :

- le programme devient difficile à lire ;
- il est compliqué à corriger ou à modifier ;
- certaines parties de code sont répétées inutilement.

La **modularité** consiste à **découper un programme en parties logiques indépendantes**, appelées **modules**, afin d'améliorer la clarté, la maintenance et la réutilisation du code.

En Java, il est possible de faire de la modularité **sans utiliser la programmation orientée objet**.

6.1 Qu'est-ce qu'un module ?

Un **module** est une partie autonome d'un programme qui :

- réalise une tâche précise ;
- peut être réutilisée ;
- peut être testée indépendamment.

Un module reçoit généralement :

- des **données en entrée** ;
 - effectue un **traitement** ;
 - produit éventuellement un **résultat**.
-

6.2 Pourquoi modulariser un programme ?

La modularité permet :

- de découper un problème complexe en sous-problèmes simples ;
- d'éviter la duplication de code ;

- de faciliter la lecture et la compréhension ;
 - de rendre le programme plus facile à maintenir ;
 - de mieux préparer le passage à la POO.
-

6.3 La méthode comme unité de modularité

En Java, la première forme de modularité repose sur les **méthodes**.

Exemple simple

```
public static int somme(int a, int b) {  
    return a + b;  
}
```

Cette méthode est un module car :

- elle a une responsabilité unique (calculer une somme) ;
 - elle peut être réutilisée plusieurs fois ;
 - elle est indépendante du reste du programme.
-

6.4 Procédure et fonction

Procédure

Une procédure est une méthode qui **ne renvoie rien**.

```
public static void afficherMessage(String msg) {  
    System.out.println(msg);  
}
```

Fonction

Une fonction est une méthode qui **renvoie une valeur**.

```
public static int carre(int n) {  
    return n * n;  
}
```

6.5 Types de modules en Java (approche procédurale)

Il existe **plusieurs types de modules** en Java avant la POO.

6.5.1 Modules simples (méthodes isolées)

Ce sont des méthodes déclarées dans la classe principale.

```
public class Main {  
  
    public static int max(int a, int b) {  
        return (a > b) ? a : b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(max(5, 9));  
    }  
}
```

Avantage : simple à comprendre.

Inconvénient : le fichier devient rapidement surchargé.

6.5.2 Modules regroupés dans une classe utilitaire

On regroupe les méthodes liées dans une **classe utilitaire**.

Fichier : `UtilsMath.java`

```
public class UtilsMath {  
  
    public static int somme(int a, int b) {  
        return a + b;  
    }  
  
    public static int max(int a, int b) {  
        return (a > b) ? a : b;  
    }  
  
    public static boolean estPair(int n) {  
        return n % 2 == 0;  
    }  
}
```

Utilisation

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(UtilsMath.somme(4, 6));  
        System.out.println(UtilsMath.estPair(10));  
    }  
}
```

Cette classe joue le rôle d'un **module procédural**.

6.5.3 Modules de traitement de tableaux

Fichier : `UtilsTableau.java`

```
public class UtilsTableau {  
  
    public static int somme(int[] T) {  
        int s = 0;  
        for (int v : T) {  
            s += v;  
        }  
        return s;  
    }  
  
    public static int max(int[] T) {  
        int m = T[0];  
        for (int v : T) {  
            if (v > m) m = v;  
        }  
        return m;  
    }  
}
```

Ces méthodes sont indépendantes de `main` et peuvent être réutilisées dans plusieurs programmes.

6.5.4 Modules de validation de données

Un module peut être utilisé pour **contrôler les données saisies**.

```
public class UtilsValidation {  
  
    public static boolean estEntre(int valeur, int min, int max) {  
        return valeur >= min && valeur <= max;  
    }  
}
```

Ce type de module permet de centraliser les règles de validation.

6.6 Organisation modulaire d'un programme

Un programme bien structuré suit généralement cette organisation :

- une classe principale avec `main` ;
- plusieurs classes utilitaires (modules) ;
- chaque module dans son propre fichier.

Exemple :

```
Main.java
UtilsMath.java
UtilsTableau.java
UtilsValidation.java
```

6.7 Rôle de la méthode `main` dans un programme modulaire

Dans un programme modulaire, la méthode `main` :

- ne contient presque aucun calcul ;
- appelle les modules ;
- coordonne l'exécution du programme.

```
public class Main {
    public static void main(String[] args) {
        int[] T = {2, 5, 8, 1};
        System.out.println(UtilsTableau.max(T));
    }
}
```

6.8 Passage de paramètres et retour de résultats

Un module peut :

- recevoir des paramètres simples (int, double) ;
- recevoir des structures (tableaux) ;
- renvoyer un résultat ou non.

```
public static double moyenne(int[] T) {
    return (double) UtilsTableau.somme(T) / T.length;
}
```

6.9 Modularité et gestion des erreurs

La modularité facilite la gestion des erreurs.

```
public static int division(int a, int b) {  
    if (b == 0) {  
        System.out.println("Erreur : division par zéro");  
        return 0;  
    }  
    return a / b;  
}
```

Chaque module protège sa propre logique.

6.10 Bonnes pratiques de modularité procédurale en Java

- Une méthode = une responsabilité
 - Noms explicites des méthodes
 - Méthodes courtes
 - Pas de calcul complexe dans `main`
 - Modules réutilisables
 - Paramètres clairement définis
-

6.11 Lien entre modularité et POO

La modularité procédurale prépare naturellement à la POO :

- une classe utilitaire devient une classe métier ;
- une méthode devient un comportement d'objet ;
- les données seront plus tard intégrées dans des objets.

Le Chapitre 6 est donc une **transition logique** vers le Chapitre 7.

Conclusion du chapitre

La modularité permet :

- d'écrire des programmes clairs et organisés ;
- de résoudre des problèmes complexes étape par étape ;
- de préparer efficacement l'apprentissage de la programmation orientée objet.

La programmation orientée objet est une **extension naturelle** de cette modularité.